Step 2 Comprehension Check

1. Question 1
    a. DECIMAL(10,2) store values into 2 decimal places. This prevents potential rounding errors from more decimal places. Since money is usually represented with 2 decimal places, this is better than a floating point which is represented with more than 2 decimal places. Floating point values could impose alight approximations depending on calculation which could introduce new issues with salaries.
    b. WHERE filters before the grouping operation is performed. Having filters after grouping has occurred. Grouping is utilizing functions such as AVG() or SUM(). HAVING can utilize aggregate functions inside of the previously mentioned functions.
    c. A PRIMARY KEY can guarantee that each row has a unique identifier preventing possible duplicate records. PRIMARY KEY also cannot be NULL. AUTO_INCREMENT assigns IDs to the next available, eliminating manual entry and possible human error. AUTO_INCREMENT also allows for more efficient ID generation.
    d. These each have a priority in which theyre executed From > Where > Group BY > SELECT > ORDER BY. This assists in potential bug fixing. This also explains why the GROUP BY functions cannot be used in the WHERE functions because grouping has not yet occurred. These help understand execution order and why certain other functions dont work in certain orders more generally.
2. Question 2

```
MariaDB [cvatter1]> SELECT title, author, price FROM BOOKS WHERE price >= 10.00 ORDER BY price DE
SC;
+---------------------+----------------+-------+
| title               | author         | price |
+---------------------+----------------+-------+
| Stewie's Adventures | Stewie Griffin | 14.99 |
| Stewie's Adventures | Stewie Griffin | 14.99 |
| Stewie's Adventures | Stewie Griffin | 14.99 |
| Lois's Recipes      | Lois Griffin   | 10.99 |
| Lois's Recipes      | Lois Griffin   | 10.99 |
| Lois's Recipes      | Lois Griffin   | 10.99 |
+---------------------+----------------+-------+
6 rows in set (0.000 sec)
```

This query is looking for title, author, and price from the inputted books. I have multiple copies of each book because I refreshed the web page creating multiple entries just to test. It also has a price condition which is why each book selected has a price of over 10.00.

```
MariaDB [cvatterl]> SELECT author, COUNT(*) AS total_books, AVG(price) AS avg_price FROM BOOKS GROUP BY author HAVI
NG AVG(price) >= 10.00 ORDER BY avg_price DESC;
+----------------+-------------+--------------------+
| author         | total_books | avg_price          |
+----------------+-------------+--------------------+
| Stewie Griffin |           3 | 14.989999771118164 |
| Lois Griffin   |           3 | 10.989999771118164 |
+----------------+-------------+--------------------+
2 rows in set (0.000 sec)
```

This is a similar query, but it only asks for the author counts and renames that new category total_books. It also retains the 10.00 price. This utilizes the GROUP BY function to group the results by author. It also uses COUNT(*) and AVG() to sum up all books and average price.

3. Step 2 SQL understanding changed my plan for step 3 PHP CRUD by demonstrating different ways to modify user inputs. CRUD stands for create, read, update, delete and this SQL section showed me ways to create data/tables and read/select data from those tables. From what I remember from another course, SQL can also manually update different datas inside the created tables using UPDATE. Additionally, SQL can be used to delete rows, cells, columns, or entire tables using DELETE. Overall, Step 2 SQL demonstrated the tools that I can use for CRUD related activities.

CRUD

URL: https://codd.cs.gsu.edu/~cvatter1/WP/IC/9/employee_demo.php

Reflection: Regarding security, prepared statements separate a SQL code from what a user may input. This could stop attackers from changing queries with malicious data. These prepared statements also help keep databases containing sensitive data more secure (business data, passwords, personal info, financial info, etc.). The prepared statements also improve query reliability by making them more consistent in formatting or other input errors. Additionally, the bind_param() function helps SQL queries receive user inputs safely because it binds variables to the prepared statements and ensures that the inputted information is treated as a data rather than an executable SQL code.

```
MariaDB [cvatter1]> USE cvatter1;
Database changed
MariaDB [cvatter1]> SHOW TABLES;
+--------------------+
| Tables_in_cvatter1 |
+--------------------+
| BOOKS              |
| STUDENTS           |
| employees          |
+--------------------+
3 rows in set (0.000 sec)

MariaDB [cvatter1]> DESCRIBE employees;
+----------+---------------+------+-----+---------+----------------+
| Field    | Type          | Null | Key | Default | Extra          |
+----------+---------------+------+-----+---------+----------------+
| id       | int(11)       | NO   | PRI | NULL    | auto_increment |
| emp_name | varchar(50)   | NO   |     | NULL    |                |
| job_name | varchar(50)   | NO   |     | NULL    |                |
| salary   | decimal(10,2) | NO   |     | NULL    |                |
+----------+---------------+------+-----+---------+----------------+
4 rows in set (0.001 sec)

MariaDB [cvatter1]> SELECT COUNT(*) AS total FROM employees;
+-------+
| total |
+-------+
|     1 |
+-------+
1 row in set (0.001 sec)

MariaDB [cvatter1]> SELECT * FROM employees ORDER BY emp_id DESC LIMIT 5;
ERROR 1054 (42S22): Unknown column 'emp_id' in 'order clause'
```

Critical Thinking Questions

1. MySQLi is preferred over older MySQL extensions because of its better security. For example, MySQLi supports prepared statements which have better security features compared to older version of MySQL. An instance of prepared statements being more secure is when dealing with SQL injections because it helps prevent these attacks. MySQLi also has newer feature (intuitively since its newer) and has more enhanced database interaction methods/functions. As well as functionality, MySQLi has better error handling with more built in error reporting, making bug fixing easier.

   Another benefit of MySQLi is that it is more reliable. Enhanced connection checks help prevent more abstact errors from causing errors. To clarify, the errors are more obvious than older versions, making fixes easier to implement. Additionally, since MySQLi is newer, its more maintainable and integrated with other modern applications.

   INSERT INTO employees (emp_name) VALUES (‘Sample Name’);

2. Inserting data using raw SQL strings directly combines a user input with an SQL command to add/modify an existing database (or creating/deleting a database). Though this makes the inputs easier to handle, it also creates security risks. A notable example of a security risk is an SQL injection. This is where an attacker can insert their own SQL code into input fields that can modify databases in an unintended way.

   Prepared statements decouple an user input and a SQL logic. This provides additional security because with the input fields not being directly converted to queries, SQL injections are easier to defend against, increasing security. Another benefit of prepared statements is improved reliability for queries. This is because the prepared statement can be reused and can filter unexpected inputs, reducing query errors.

   $sql = "INSERT INTO employees VALUES ('$name', '$job')";

3. I would keep it in one table. I would do this because it keeps the different data together easier, resulting in simpler queries since all data is stored in a single location. Though the one table could cause redundant information, overall the simple queries are enough to get me to want to stick to a single table.

Normalizing the tables/data would reduce some of this potential duplicate information because the information would be better separated. Though the redundant information would still exist, the readability would be greatly increased since that data would be separated. With the information be separated into multiple tables, the chances of information being accidentally accessed is reduced, making the normalized tables/data safer in that sense. Though this does greatly increase the complexity of the queries.

CREATE TABLE employees (empid INT PRIMARY KEY, empname VARCHAR(50), department_name VARCHAR(50));

4. The order I would go with is, connection test > SQL test > CRUD test > UI test. I'm choosing this order because the connection test will verify that the PHP that I am utilizing can successfully connect to MySQLi. This is critical because without that, the rest of the program would not function. Next, the SQL test. After verifying that it connects correctly I need to check if the queries work correctly. This is because even if the SQL is connected, if the queries don't work correctly, then the connection is pointless.

   Next, I would do the CRUD test. This is a more “advanced” version of the SQL test because it tests creating, reading, updating, and deleting information from the tables rather than just retrieving the information. This could be interchanged with the SQL test because theyre similar, but I think of the SQL test as a baseline, so if I cant event gather information what chance is there to modify it. Finally, I would do the UI test. This is the user interface which is very important, but without functionality the user interface is pointless. Its in the same line of thinking as “function over form”. This is because its better to have a bad looking website rather than a badly working website.

   $conn = new mysqli($host, $user, $pass, $db);
   if ($conn->connect_error) { echo "Connection failed"; }

5. WHERE is critical for an UPDATE/DELETE because it limits/filters which parts of a table are being modified. The WHERE indicates which rows/cols the user intendes to update or delete. Without WHERE all rows in a table are affected. This can cause massive changes without that being the intention when UPDATE is used. Or in a worse situation, DELETE can delete all rows.

If one of these situations occur, the data integrity of that table can be destroyed because with a single query important user data could be deleted. Though this data may be recoverable, it may also take a long amount of time. Additionally, once the data is recovered it could still leave the data integrity lower than before the mistaken query.

WITHOUT WHERE

UPDATE employees SET salary = 50000;

With WHERE

UPDATE employees SET salary = 50000 WHERE emp_id = 1;

6. Improving the aspects of a database of a large size to improve query times is very important because it reduces “overhead” and unnecessary work done by the computer, it also reduces performance issues as the database grows further in size. Adding indexes could improve query speed because it allows the database to find specific records more efficiently instead of scanning each row in the table.

   Implementing pagination could also improve query speeds by loading only necessary sections instead of thousands and thousands of records all at once. This improves both application speed and the user experience while querying. Also, improving table structure can also help by reducing redundancy and improving data consistency. Though these factors can be helpful with speed improvements, modifying these aspects after large amounts of data are already in the database could cause unintended problems because large sweeping changes being done to large amounts of data is bound to cause issues.

   CREATE INDEX idx_employee_name ON employees(emp_name);

   SELECT * FROM employees WHERE emp_name = 'SAMPLE NAME’;

7. Having a multi-page structure can improve debugging efficiency by having an organized structure. This is critical because it helps separate different topics into different pages, making information easier to locate and lowering the overwhelming nature of large amounts of data. This is in the same line as separate large programs into sub functions/methods or creating different objects in OOP languages (with a

common exception for Python). This well organized structure also adds clear navigation because with the data being well separate finding that data becomes easier as compared to sifting through all the data at once.

Since many of the datas could be organized the same way, many of the code blocks could be copyable code blocks. This makes debugging easier since the errors should be easier to find. This will also improve student confidence since the code/data is easier to locate and fix.

SELECT * FROM employees WHERE department_name = 'Engineering';

This is an example of a code block that could be copied in many spots and where having all data in the same table could cause issues or force more lengthily and complex queries.

8. Some specific feedback I would give my own project after testing it end-to-end would be that after checking the entire process starting the data entering, PHP processing, then storing in MySQLi and displaying results, I should try to identify potential reliability issues or weak points in the code. This means that if there are any sections that are fragile (could accept faulty data or are more vulnerable to attacks) and try to enhance them. I would also add further error handling into the code.

   Out of these, identifying the fragile sections of my code and enhancing them to better handle these scenarios would be the best improvement. This is because if the website is accepting faulty inputs then it is not reliable, so fixing that issue would most increase reliability. Additionally, this is compared to security, though security is important, having the website being usable is far more important when it comes to reliability.

   INSERT INTO employees (emp_name, job_name, salary, emp_id) VALUES ('SAMPLLE NAME', 'Developer', 80000.00, 01010204);

   This includes an employee id which can help identify the employee and potentially prevent the wrong person from being pulled from the database on query. This improves usability because it could remove potential faulty inputs or vague queries.

9. Originally when debugging SQL, or PHP I would just run a query or the program and go to the line that there was an issue at. This could cause problems because maybe the issue does not stem from that specific part, but from another larger section of code. This also disreguards the priority each section of code has and means I could be looking in the entirely wrong section of my code.

   After completing this assignment I would start from the highest priority test and work my way down so that I can ensure each section is working correctly before moving on. This changes debugging into more of a checklist rather than a free for all. Out of all these critical-thinking lessons I think that the question “What testing sequence best prevents last-minute failres: connection test, SQL test, CRUD test, or UI test first? Defend your order.” is the most necessary to document in my Word file because its more than likely the most important for debugging. This is because (as I stated earlier) it turns debugging into more of a checklist that can be easily followed rather than just jumping around the code trying to solve the error.

   SELECT * FROM employees;

   This query demonstrates the ability to verify that the table structures are working correctly for the CRUD operations in the database.